RO015

Hyperparameter Optimisation of Vision Transformers (ViTs) to Identify Malignant Colorectal Tumours

1 Overview of the Project

1.1 Literature Review

Cancer is one of the leading causes of death globally, accounting for an estimated 9.6 million deaths, or 1 in 6 deaths, in 2018 [1]. The burden for treating cancer continues to grow, resulting in physical and financial strain on affected parties as well as healthcare systems. While there is no universal cure for cancer, research has shown that the cancer burden can effectively be reduced through the early detection of cancer when the disease has not spread [2].

Hence, there is a rising need to develop tools to aid in the early detection of cancer. Recent developments of Artificial Intelligence (AI) have boosted progress in this area, particularly in the field of machine learning (ML). Rapid developments in ML have resulted in the proliferation of new models such as the Vision Transformer (ViT) for image classification [3], contesting state-of-the-art Convolutional Neural Network (CNN) models. Such new technology has also been applied in the healthcare industry, as seen by the use of ViT models to identify lung cancer [4]. Some models have been largely successful, attaining accuracies of as high as 96% [5], a groundbreaking achievement which could significantly improve healthcare in the near future.

Yet, pretrained models are not a one-size-fits-all solution – many limitations still exist with the usage of pretrained models. Firstly, pretrained deep learning models are computationally intensive [6], often requiring multiple GPUs for training [4]. Additionally, pretrained models are often relatively inflexible, especially if the model was trained on a largely different dataset to solve another problem [7]. Hence, various problems still exist with directly applying pretrained machine learning models to solve general tasks today.

1.2 Rationale

As such, this project aims to develop a simple, yet comprehensive and complex ViT model from scratch with finetuned hyperparameters, trained on a small dataset to identify malignant tumours with reasonable accuracy, aiding in the early detection of cancer through the use of AI.

2 Aims of the Project

1. Develop a simple and effective ViT model from scratch which can be trained with little computational power.

2. By fine tuning hyperparameters, achieve reasonable accuracy and F1 score on identifying malignant tumours, and compare the results with other models such as CNNs.

3 Terminology

1. ViT: Vision Transformer, a machine learning model which utilises the key idea of self-attention to learn.

2. CNN: Convolutional Neural Network, a machine learning model which learns feature engineering by itself through filters or kernel optimization.

4 Methodology

1. Develop a commputationally efficient ViT model, and tune its hyperparameters

2. Develop a standard CNN model with tuned hyperparameters

3. Run the 2 models, as well as a pretrained ViT model and CNN model on the dataset and compare the results.

4.1 Preparation of the Dataset

A small public dataset of 165 H&E stained colorectal cancer images was obtained from the MICCAI society through a gland segmentation competition [8], and the dataset was acquired by a team of pathologists at the University Hospitals Coventry and Warwickshire [9]. This dataset consists of a roughly even split of malignant and benign images. To further enhance the accuracy of the model, the dataset was augmented with transformations, such as random rotation, random horizontal flipping and blurring of the images. The final dataset consisted of 395 training images, 85 validation images and 80 test images to form a total of 560 images.

A sample of such images are shown below.

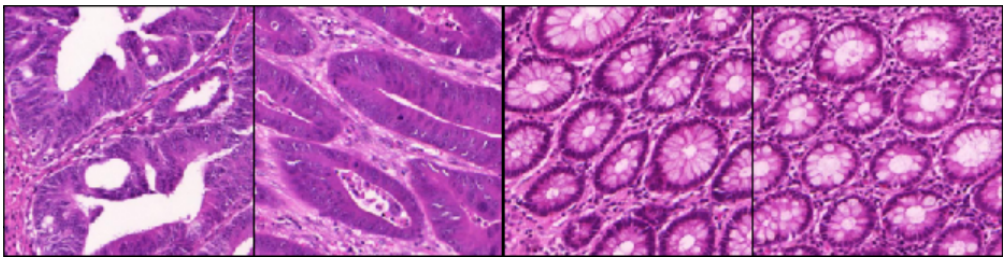


Figure 1: The first two images represent malignant tumours, and the other two benign tumours.

4.2 Development of the ViT model

4.2.1 Model Architecture

ViTs are an extended version of transformers, revolving around the concept of self-attention [10], a mechanism which allows the model to identify and weigh the importance of different parts of the input sequence [11], and is used for tasks such as image classification and object detection.

The model first does patch embedding, splitting an image into various patches with a certain patch size. After which, a learnable token is added to each patch for the model to note its position in the image. Then, the patches are passed through a transformer encoder which uses self attention to process the patches. Lastly, the patches are passed through a multi-layered perceptron to determine the image class. A diagram of the model architecture is shown below, and a ViT model was implemented using pytorch in Appendix 9.1. Specifically, 12 blocks were used in the transformer encoder.

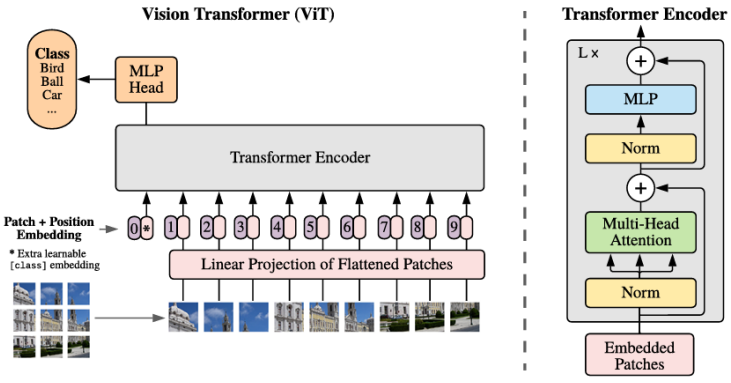


Figure 2: ViT Model Architecture [3]

4.2.2 Training the model

The default learning rate used was 10^{-5} and the default dropout rate was 0.4. However, in order for the ViT [3] to train successfully with enhanced acccuracy, hyperparameters should be optimised in addition to the existing model architecture for the model to learn effectively. Hyperparameters are external variables which are preset before the start of model training, affecting model parameters during training [12]. Such hyperparameters include the learning rate, dropout rate and patch size. Additionally, a time-based scheduler and the Adam optimiser was used in training for its ability to learn quickly.

To determine the optimal hyperparameters for this classification problem, various optimisation methods were used, such as the grid search method on a small range of hyperparameter values. This method involves training the model on all different possible combinations of hyperparameters, and setting the optimal hyperparameters to those which minimise the validation loss.

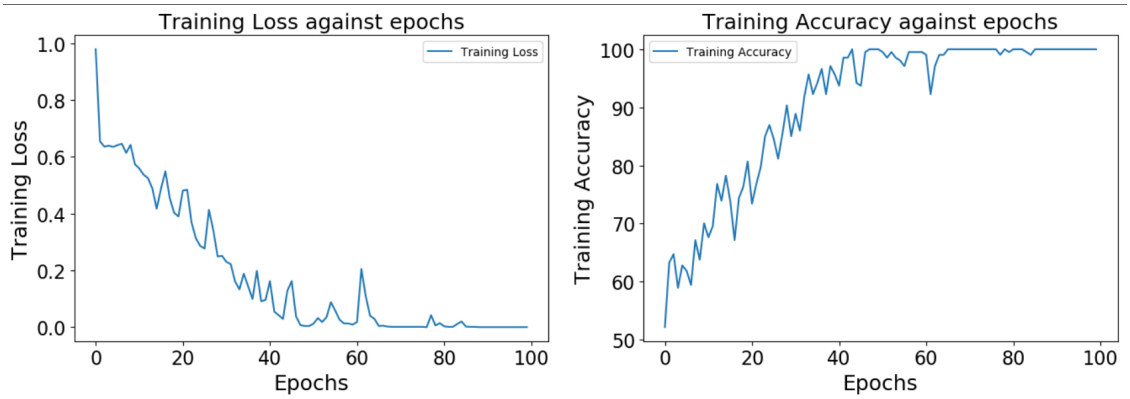
The list of hyperparameters considered is shown in the table below.

Hyperparameter	List of Values	Description
Learning Rate	$10^{-8}, 10^{-7}, 10^{-6}, 10^{-5}, 10^{-4}, 10^{-3}$	Determines how fast the model learns
Dropout rate	$0.0, 0.1, 0.2, 0.3, 0.4, 0.5$	Prevents overfitting

Results of the grid search algorithm has shown that the optimal learning rate is 10^{-5} and the optimal dropout rate is 0. After which, the model was trained on the full training dataset for 100 epochs.

Additionally, the Bayesian optimisation method, an optimisation algorithm for objective functions which are computationally intensive to calculate [13], was used to look for any combination of hyperparameters which could further improve the results. To do so, previous results from the grid search were used to narrow down the search range for the learning rate to between 10^{-6} and 10^{-4}, while the search range for the dropout rate was narrowed down to between 0 and 0.1.

The Bayesian optimisation algorithm was run for 50 iterations to determine the hyperparameters which minimised the validation loss. After which, the optimal learning rate obtained was $1.65 \cdot 10^{-5}$ and the optimal dropout rate obtained was 0.081. The model was then trained for 100 epochs with these hyperparameters, and the training curves can be shown on the next page. The code can be found in Appendix 9.5.

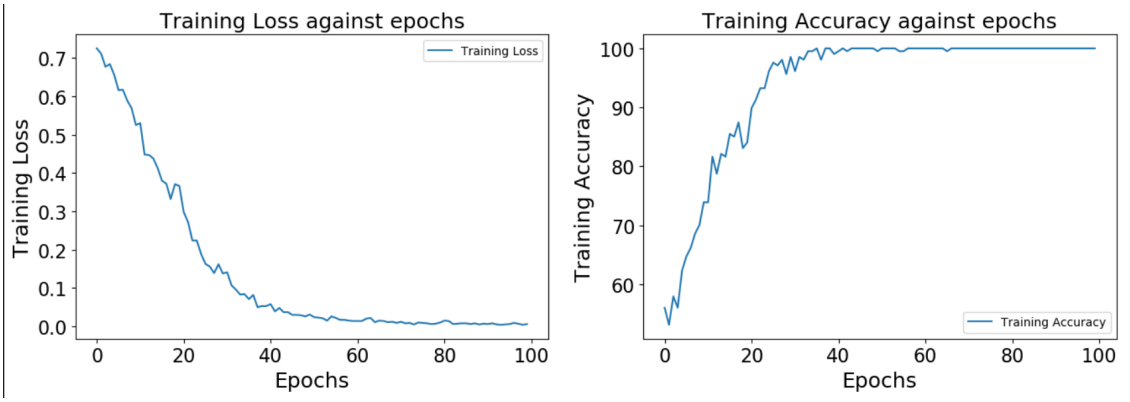


The entire code for training the ViT model can be found in Appendix 9.6.

4.3 Development of a standard CNN model

To better compare the results of the ViT [3] model, a standard CNN model was also developed to solve the problem. This model utilised 2 convolutional and pooling layers to extract key features, having a kernel size of 3x3. After which, a multi-layered perceptron layer was added to perform the classification task. The architecture of the CNN can be found in Appendix 9.2 and code can be found in Appendix 9.3. Similar to the ViT [3], the CNN was also trained with the hyperparameter optimisation algorithms.

After grid search, the optimal learning rate was found to be 10^{-5} and the optimal dropout rate was found to be 0.1. Then, the Bayesian optimisation algorithm gave an optimal learning rate of $5.31 \cdot 10^{-5}$ and an optimal dropout rate of 0.161. The model was trained on these hyperparameters for 100 epochs. The training curves for the CNN can be shown below.



4.4 Pretrained models

Moreover, some pretrained models were also used to better determine the effectiveness of the ViT model [3] with tuned hyperparameters in identifying tumours. This includes timm's pretrained ViT model, vit_base_patch16_224, as well as AlexNet, a popular pretrained CNN used for classification tasks. These models were trained with the default hyperparameters, setting the learning rate to be 10^{-5} and the dropout to be 0.4.

5 Results

ID	Model	Optimisation	Test Accuracy	F1 score
1	ViT trained from scratch	Grid Search	$72.9 \pm 1.1\%$	0.739 ± 0.009
2	ViT trained from scratch	Bayesian	$72.5 \pm 0.0\%$	0.744 ± 0.000
3	ViT trained from scratch	default	$60.0 \pm 0.0\%$	0.686 ± 0.000
4	CNN trained from scratch	Grid Search	$62.5 \pm 1.4\%$	0.706 ± 0.008
5	CNN trained from scratch	Bayesian	$63.8 \pm 0.0\%$	0.690 ± 0.002
6	CNN trained from scratch	default	$57.5 \pm 0.7\%$	0.693 ± 0.004
7	timm vit_base_patch16_224	default	$68.8 \pm 0.0\%$	0.757 ± 0.000
8	AlexNet CNN	default	**75.0 ± 0.0%**	**0.773 ± 0.000**

*Results are rounded off to 3s.f.

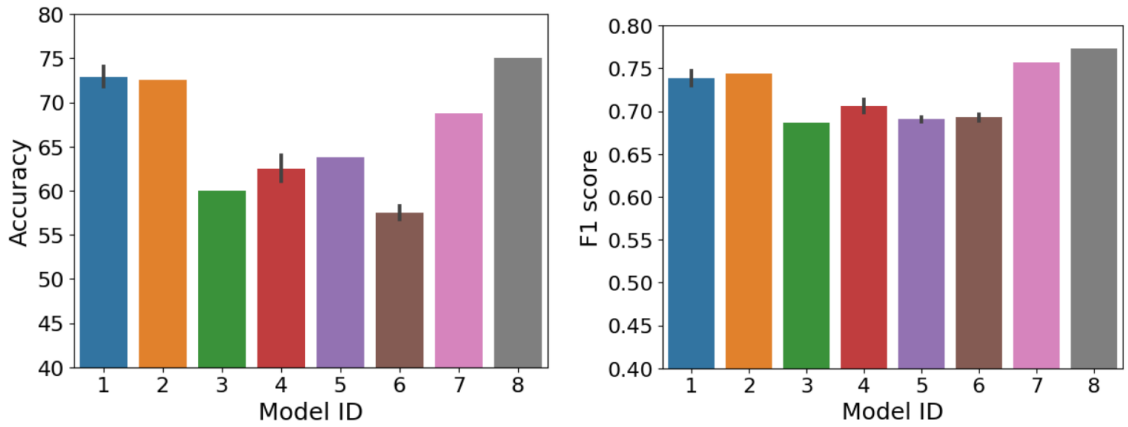


Figure 3: Barplot of the models' performance

As seen, the pretrained models perform slightly better than the models trained from scratch in terms of F1 score. The AlexNet CNN model achieved the best results in both metrics, but the ViTs [3] trained from scratch, especially with tuned hyperparameters, achieved comparable results. The accuracy obtained is highly reasonable for a model trained on a small dataset of 560 images, and it also surpasses the accuracy of the pretrained ViT model.

Additionally, the results have shown that hyperparameter optimisation algorithms have substantially improved model performance in both the ViT [3] and the CNN.

Between the ViT [3] and the CNN, the ViT [3] performs slightly better in accuracy and F1 score as compared to the CNN. This is especially so with the hyperparameter optimisation algorithms implemented, which greatly improved the performance of the ViT [3] while improving the performance of the CNN to a smaller extent.

6 Conclusion

In conclusion, while the models trained from scratch with default parameters were relatively lacklustre, the hyperparameter optimisation algorithms significantly improved the performance of the models trained from scratch, allowing their results to be comparable to those of the pretrained models.

7 Limitations

7.1 Size of dataset

It is hard to obtain suitable data of colorectal cancer images. Hence, the size of the original dataset is relatively small, which could result in overfitting during training. While data augmentation was carried out to alleviate this, the images are still relatively similar to each other, compromising the accuracy of the model.

7.2 Multi-class Classification

Identifying benign and malignant tumours is a binary classification problem. For other tasks which might involve the classification of multiple classes, the machine learning model might not be able to achieve as high an accuracy and F1 score with the same values of hyperparameters used in training the model for this problem.

8 Future Work

To further enhance the scope of this project, experiments could be conducted on different problems, such as those with multiple classes, to determine the optimal values for hyperparameters in multi-class classification.

References

[1] *Cancer*. URL: https://www.who.int/health-topics/cancer#tab=tab_1.

[2] Mar. 2023. URL: https://www.cancerresearchuk.org/https%3A//www.cancerresearchuk.org/about-cancer/spot-cancer-early/why-is-early-diagnosis-important.

[3] Alexey Dosovitskiy et al. "An image is worth 16x16 words: Transformers for image recognition at scale". In: *arXiv:2010.11929* (2020).

[4] Hazrat Ali, Farida Mohsen, and Zubair Shah. "Improving diagnosis and prognosis of lung cancer using vision transformers: a scoping review". In: *BMC Medical Imaging* 23.1 (2023), p. 129.

[5] Bo Zhang, Huiping Shi, and Hongtao Wang. "Machine Learning and AI in Cancer Prognosis, Prediction, and Treatment Selection: A Critical Approach". In: *Journal of Multidisciplinary Healthcare* (2023), pp. 1779–1791.

[6] Xu Han et al. "Pre-trained models: Past, present and future". In: *AI Open* 2 (2021), pp. 225–250.

[7] Joshi Naveen. *Exploring the limits of Transfer learning*. Feb. 2020. URL: https://www.allerin.com/blog/exploring-the-limits-of-transfer-learning.

[8] Korsuk Sirinukunwattana et al. "Gland segmentation in colon histology images: The glas challenge contest". In: *Medical image analysis* 35 (2017), pp. 489–502.

[9] *GlaS@MICCAI'2015: Gland Segmentation*. URL: https://www.kaggle.com/datasets/sani84/glasmiccai2015-gland-segmentation.

[10] Ashish Vaswani et al. "Attention is all you need". In: *Advances in neural information processing systems* 30 (2017).

[11] *What is Self-attention?* URL: https://h2o.ai/wiki/self-attention/#:~:text=Self%2Dattention%20is%20a%20mechanism,and%20relationships%20within%20input%20sequences..

[12] *What is Hyperparameter Tuning?* URL: https://aws.amazon.com/what-is/hyperparameter-tuning/.

[13] Peter I Frazier. "A tutorial on Bayesian optimization". In: *arXiv:1807.02811* (2018).

9 Appendix

9.1 Code for ViT model

```
class patchembed(nn.Module):
    def __init__(self, img_size, patch_size, chans, embed_dim):
        super().__init__()
        img_size = (img_size, img_size)
        patch_size = (patch_size, patch_size)
        num_patches = (img_size[0] // patch_size[0]) ** 2
        self.img_size = img_size
        self.patch_size = patch_size
        self.num_patches = num_patches
        self.c1 = nn.Conv2d(chans, embed_dim, patch_size, patch_size)

    def forward(self, x):
        B, C, H, W = x.shape
        return self.c1(x).flatten(2).transpose(1, 2)

class self_attention(nn.Module):
    def __init__(self, embed_dim, num_heads, qkv_bias = False, qk_scale = None,
        attn_drop = 0, proj_drop = 0):
        super().__init__()
        self.num_heads= num_heads
        head_dim = embed_dim // num_heads
        self.scale = qk_scale or head_dim ** -0.5
        self.qkv = nn.Linear(embed_dim, 3 * embed_dim, bias = qkv_bias)
        self.drop1 = nn.Dropout(attn_drop)
        self.mlp = nn.Linear(embed_dim, embed_dim)
        self.drop2 = nn.Dropout(proj_drop)

    def forward(self, x):
        btch, patch_num, patch_dim = x.shape
        qkv = self.qkv(x).reshape(btch, patch_num, 3, self.num_heads, patch_dim
            // self.num_heads).permute(2, 0, 3, 1, 4)
        q, k, v = qkv[0], qkv[1], qkv[2]
        attn = (q @ k.transpose(-2, -1)) * self.scale
        attn = attn.softmax(dim = -1)
```

```
        attn = self.drop1(attn)
        res = (attn @ v).transpose(1, 2).reshape(btch, patch_num, patch_dim)
        res = self.mlp(res)
        res = self.drop2(res)
        return res

class ffn(nn.Module):
    def __init__(self, dim, hidden_features = None, dropout = 0):
        super().__init__()
        if(hidden_features == None):
            hidden_features = dim
        self.fc1 = nn.Linear(dim, hidden_features)
        self.act = nn.GELU()
        self.fc2 = nn.Linear(hidden_features, dim)
        self.drop = nn.Dropout(dropout)

    def forward(self, x):
        x = self.fc1(x)
        x = self.act(x)
        x = self.drop(x)
        x = self.fc2(x)
        x = self.drop(x)
        return x

class block(nn.Module):
    def __init__(self, dim, num_heads, mlp_ratio = 4, qkv_bias = False, qk_scale
        = None, drop = 0, attn_drop = 0, drop_path = 0, norm_layer =
        nn.LayerNorm):
        super().__init__()
        self.norm1 = norm_layer(dim)
        self.attn = self_attention(dim, num_heads, qkv_bias = qkv_bias, qk_scale
            = qk_scale, attn_drop = attn_drop, proj_drop = drop)
        self.norm2 = norm_layer(dim)
        self.mlp = ffn(dim, hidden_features = dim * mlp_ratio)
        self.drop_path = nn.Identity()

    def forward(self, x):
        x = x + self.drop_path(self.attn(self.norm1(x)))
```

```
        x = x + self.drop_path(self.mlp(self.norm2(x)))
        return x

class ViT(nn.Module):
    def __init__(self, embed_dim = 768, img_size = 224, patch_size = 16, in_chans
        = 3, num_classes = 1000, num_blocks = 12, num_heads = 12, mlp_ratio = 4,
        qkv_bias = None,
              qk_scale = None, drop = 0., attn_drop = 0., drop_path = 0.):
        super().__init__()
        self.num_classes = num_classes
        self.num_features = self.embed_dim = embed_dim

        self.patch_embed = patchembed(img_size, patch_size, in_chans, embed_dim)
        num_patches = self.patch_embed.num_patches

        self.cls_token = nn.Parameter(torch.zeros(1, 1, embed_dim)) # first 1 is
            the batch size
        self.pos_embed = nn.Parameter(torch.zeros(1, num_patches + 1, embed_dim))
        self.pos_drop = nn.Dropout(drop)

        depth_decay = [x.item() for x in torch.linspace(0, drop_path, num_blocks)]

        self.encoders = nn.Sequential(
            *[block(dim = embed_dim, num_heads = num_heads, mlp_ratio =
                mlp_ratio, qkv_bias = qkv_bias, qk_scale = qk_scale,
                      drop = drop, attn_drop = attn_drop, drop_path =
                          depth_decay[i], norm_layer = nn.LayerNorm) for i in
                          range(num_blocks)]
        )

        self.norm = nn.LayerNorm(embed_dim)
        self.pre_logits = nn.Identity()

        self.mlphead = nn.Linear(embed_dim, num_classes)

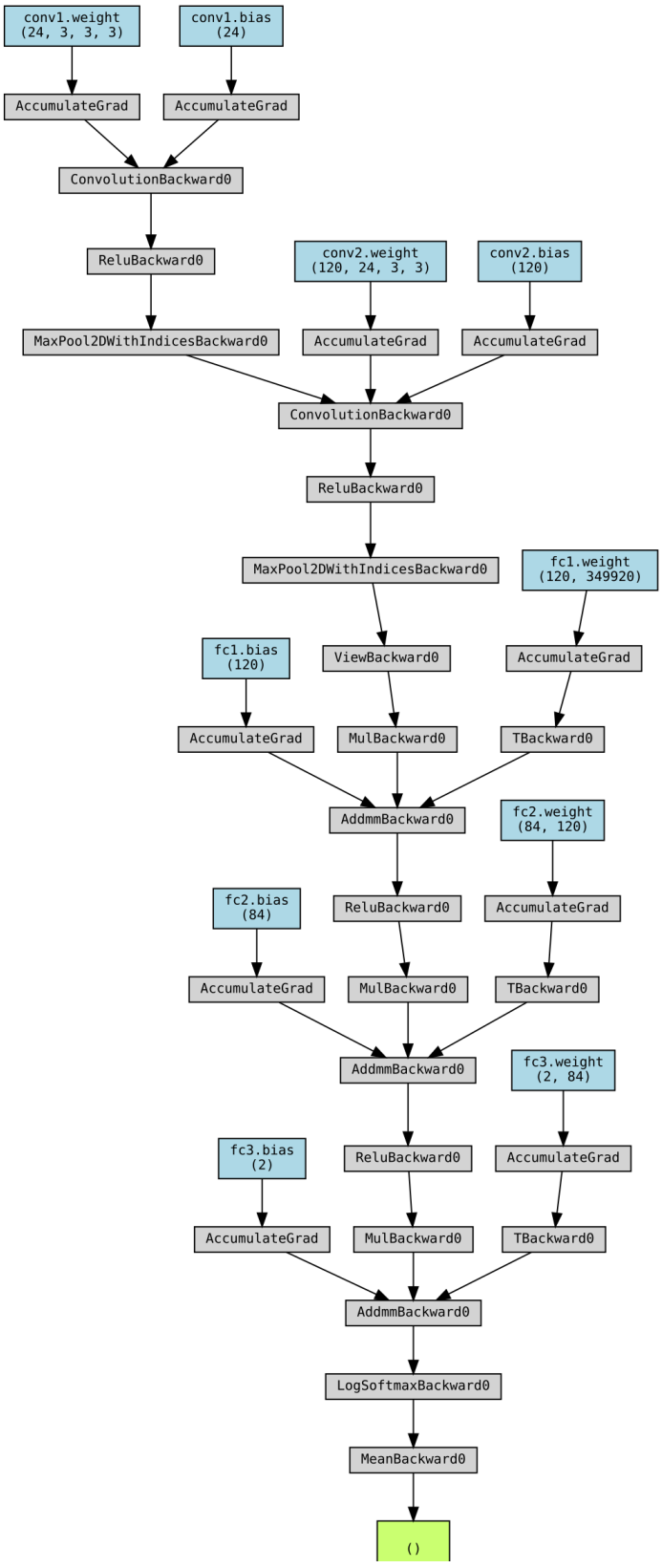
    def forward_features(self, x):
        x = self.patch_embed(x)
        cls_token = self.cls_token.expand(x.shape[0], -1, -1)
```

```
        x = torch.cat((cls_token, x), dim = 1)
        x = self.pos_drop(x + self.pos_embed)

        x = self.encoders(x)
        x = self.norm(x)
        return x

    def forward(self, x):
        x = self.forward_features(x)
        x = self.pre_logits(x[:, 0])
        x = self.mlphead(x)
        return x
```

9.2 Architecture for CNN model



9.3 Code for CNN model

```
class CNN(nn.Module):
    def __init__(self, drp = 0.4):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 24, 3, 1)
        self.conv2 = nn.Conv2d(24, 120, 3, 1)
        self.fc1 = nn.Linear(54*54*120, 120)
        self.dropout = nn.Dropout(drp)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 2)

    def forward(self, X):
        X = F.relu(self.conv1(X))
        X = F.max_pool2d(X, 2, 2)
        X = F.relu(self.conv2(X))
        X = F.max_pool2d(X, 2, 2)
        X = X.view(-1, 54*54*120)
        X = self.dropout(X)
        X = self.dropout(F.relu(self.fc1(X)))
        X = self.dropout(F.relu(self.fc2(X)))
        X = self.fc3(X)
        return F.log_softmax(X, dim=1)
```

9.4 Code for Grid Search

```
epochs = 50
lr = 10 ** (-7)
drp = 0.0
ptch_sz = 16
lr_pos = [10 ** (-8), 10 ** (-7), 10 ** (-6), 10 ** (-5), 10 ** (-4), 10 ** (-3)]
drop_pos = [0, 0.1, 0.2, 0.3, 0.4, 0.5]
patch_pos = [16]
def grid_search(lr_tmp, drop_tmp, patch_tmp):
    max_trn_batch = 10
    model = ViT(num_classes = 2, drop = drop_tmp, attn_drop = 0, drop_path =
        drop_tmp, patch_size=patch_tmp)
```

```
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr_tmp, betas=(0.9,
        0.999))
    min_r = 0.5
    s_func = lambda epoch: 0.5 * (1+math.cos(epoch*math.pi/epochs)) * (1-min_r) +
        min_r
    scheduler = torch.optim.lr_scheduler.LambdaLR(optimizer, lr_lambda=s_func)
    for epoch in range(epochs):
        model.train()
        # b is the batch number
        for b, (X_train, y_train) in enumerate(train_loader):
            b+=1
            if(b == max_trn_batch):
                break
            optimizer.zero_grad()
            y_pred = model.forward(X_train)
            loss = criterion(y_pred, y_train)
            loss.backward()
            optimizer.step()
        scheduler.step()
    model.eval()
    with torch.no_grad():
        loss_sum = 0
        for b, (X_val, y_val) in enumerate(val_loader):
            b += 1
            optimizer.zero_grad()
            y_pred = model.forward(X_val)
            loss = criterion(y_pred, y_val)
            loss_sum += loss
    return float(loss_sum / b)

min_loss = 1
for lr_tmp in lr_pos:
    for drop_tmp in drop_pos:
        for patch_tmp in patch_pos:
            ls_tmp = grid_search(lr_tmp, drop_tmp, patch_tmp)
            print(ls_tmp, lr_tmp, drop_tmp)
            if(ls_tmp < min_loss):
```

```
                min_loss = ls_tmp
                lr, drp, ptch_sz = lr_tmp, drop_tmp, patch_tmp
```

9.5 Code for Bayesian Optimisation

```
epochs = 50
lr = 10 ** (-7)
drp = 0.0
ptch_sz = 16
space= {
    'learning_rate': hp.loguniform('learning_rate', np.log(10 ** (-8)), np.log(10
        ** (-3))),
    'dropout' : hp.uniform('dropout', 0, 0.8, 0.01),
    'patch_size' : hp.choice('patch_size', [8, 16, 32])
}
def bayesian(ps):
    lr, dr, pc = ps['learning_rate'], ps['dropout'], int(ps['patch_size'])
    max_trn_batch = 10
    model = ViT(num_classes = 2, patch_size = pc, drop = dr, attn_drop = 0,
        proj_drop = dr)
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr, betas=(0.9, 0.999))
    min_r = 0.5
    s_func = lambda epoch: 0.5 * (1+math.cos(epoch*math.pi/epochs)) * (1-min_r) +
        min_r
    scheduler = torch.optim.lr_scheduler.LambdaLR(optimizer, lr_lambda=s_func)
    for epoch in range(epochs):
        model.train()
        # b is the batch number
        for b, (X_train, y_train) in enumerate(train_loader):
            b+=1
            if(b == max_trn_batch):
                break
            optimizer.zero_grad()
            y_pred = model.forward(X_train)
            loss = criterion(y_pred, y_train)
```

```
            loss.backward()
            optimizer.step()
        scheduler.step()
    model.eval()
    with torch.no_grad():
        loss_sum = 0
        for b, (X_val, y_val) in enumerate(val_loader):
            b += 1
            optimizer.zero_grad()
            y_pred = model.forward(X_val)
            loss = criterion(y_pred, y_val)
            loss_sum += loss
    return float(loss_sum / b)

num_eval = 50
trials = Trials()
best_param = fmin(bayesian, space = space, algo=tpe.suggest, max_evals=num_eval,
    trials=trials)
lr, drp, ptch_sz = best_param['learning_rate'], best_param['dropout'],
    best_param['patch_size']
```

9.6 Code for Training the ViT model

```
def get_data(filename, batch_size):
    torch.manual_seed(101)
    root = filename
    global train_data, val_data, test_data, train_loader, val_loader, test_loader
    train_data = datasets.ImageFolder(os.path.join(root, 'train'),
        transform=transforms.ToTensor())
    #train_data2 = datasets.ImageFolder(os.path.join(root, 'train'),
        transform=transforms.ToTensor())
    val_data = datasets.ImageFolder(os.path.join(root, 'valid'),
        transform=test_transform)
    test_data = datasets.ImageFolder(os.path.join(root, 'test'),
        transform=test_transform)
    train_loader = DataLoader(train_data, batch_size = batch_size, shuffle=True)
    val_loader = DataLoader(val_data, batch_size = batch_size, shuffle = True)
```

```
    test_loader = DataLoader(test_data, batch_size = len(test_data), shuffle=True)
    return train_data, val_data, test_data, train_loader, val_loader, test_loader

def check_dataset():
    class_names = train_data.classes
    print(class_names)
    print(f'Training images available: {len(train_data)}')
    print(f'Val images available: {len(val_data)}')
    print(f'Testing images available: {len(test_data)}')
    for images,labels in train_loader:
        break

    print('Label:', labels.numpy())
    print('Class:', *np.array([class_names[i] for i in labels]))

    im = make_grid(images, nrow=5)

    inv_normalize = transforms.Normalize(
        mean=[-0.485/0.229, -0.456/0.224, -0.406/0.225],
        std=[1/0.229, 1/0.224, 1/0.225]
    )
    im_inv = inv_normalize(im)

    plt.figure(figsize=(12,4))
    plt.imshow(np.transpose(im_inv.numpy(), (1, 2, 0)));

def train_model(learning_rate, epochs):
    model = ViT(num_classes = 2, drop = drp, attn_drop = 0, drop_path = drp,
        patch_size = ptch_sz)
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate,
        betas=(0.9, 0.999))
    min_r = 0.5
    lr_current = 0
    s_func = lambda epoch: 0.5 * (1+math.cos(epoch*math.pi/epochs)) * (1-min_r) +
        min_r
    scheduler = torch.optim.lr_scheduler.LambdaLR(optimizer, lr_lambda=s_func)
    start_time = time.time()
```

```
    train_losses = []
    val_losses = []
    train_correct = []
    val_correct = []
    global predicted
    for epoch in range(epochs):
        trn_corr = 0
        val_corr = 0
        loss_sum = 0
        model.train()
        # b is the batch number
        for b, (X_train, y_train) in enumerate(train_loader):
            b+=1
            optimizer.zero_grad()
            y_pred = model.forward(X_train)
            loss = criterion(y_pred, y_train)
            predicted = torch.max(y_pred.data, 1)[1]
            batch_corr = (predicted == y_train).sum()
            trn_corr += batch_corr

            loss.backward()
            optimizer.step()
            loss_sum += loss.item()

        loss_sum /= b
        train_losses.append(loss_sum)
        train_correct.append(trn_corr / len(train_data) * 100)
        print(f'epoch: {epoch:2} loss: {loss_sum:10.8f} \
  accuracy: {trn_corr.item()*100/(len(train_data)):7.3f}%')
        loss_sum = 0
        # Run the validation batches
        with torch.no_grad():
            model.eval()
            for b, (X_val, y_val) in enumerate(val_loader):
                b += 1
                y_ans = model.forward(X_val)
                predicted = torch.max(y_ans.data, 1)[1]
```

```
                val_corr += (predicted == y_val).sum()
                loss = criterion(y_ans, y_val)
                loss_sum += loss
        scheduler.step()

        val_losses.append(loss_sum / b)
        print(loss_sum / b)
        val_correct.append(val_corr / len(val_data) * 100)
    total_time = time.time() - start_time
    #Run on the test set
    with torch.no_grad():
        model.eval()
        tst_corr = 0
        global y_test
        for b, (X_test, y_test) in enumerate(test_loader):

         # Apply the model
            y_pre = model.forward(X_test)

        # Tally the number of correct predictions
            predicted = torch.max(y_pre.data, 1)[1]
            tst_corr += (predicted == y_test).sum()

        loss = criterion(y_pre, y_test)
    print(tst_corr / len(test_data) * 100)
    return train_losses, train_correct, val_losses, val_correct, tst_corr /
        len(test_data) * 100, total_time

def train_ViT_model():
    #t_l ---> train losses, t_c --> train corrects
    #v_l --> validation losses, v_c --> validation corrects
    t_l, t_c, v_l, v_c, t_acc, time = train_model(lr, epochs)
    train_loss.append(t_l)
    train_corrects.append(t_c)
    val_loss.append(v_l)
    val_corrects.append(v_c)
    test_acc.append(t_acc)
    times = str(timedelta(seconds=time)).split(':')
```

```
    print(f'\nDuration: {times[0]} hours, {times[1]} minutes, {times[2]}
        seconds') # print the time elapsed
    print(t_acc)

def save_results():
    ls = []
    col = ['Training Loss', 'Training Accuracy', 'Validation Loss', 'Validation
        Accuracy']

    for i in range(epochs):
        tmp = []
        tmp.append(round(train_loss[0][i], 3))
        tmp.append(round(train_corrects[0][i].item(), 3))
        tmp.append(round(val_loss[0][i].item(), 3))
        tmp.append(round(val_corrects[0][i].item(), 3))
        ls.append(tmp)

    df_ls = pd.DataFrame(ls, columns = col)
    df_ls.head()
    df_ls.to_csv("results_bayesian_vit_2.csv")

def plot_training_loss():
    plt.subplot(221)
    plt.plot(train_loss[0])
    plt.xlabel('Training Loss')
    plt.legend()

def plot_val_loss():
    plt.subplot(222)
    plt.plot(val_loss[0])
    plt.xlabel('Validation Loss')
    plt.legend()

def plot_train_acc():
    plt.subplot(223)
    train_acc = train_corrects[0]
    plt.plot(train_acc)
    plt.xlabel('Training Accuracy')
```

```
    plt.legend()

def plot_val_acc():
    plt.subplot(224)
    val_acc = val_corrects[0]
    plt.plot(val_acc)
    plt.xlabel('Validation Accuracy')
    plt.legend()

plt.figure(figsize = (12, 8))
train_ViT_model()
#save_loss()
#save_acc()
save_results()
plot_training_loss()
plot_val_loss()
plot_train_acc()
plot_val_acc()
tn, fp, fn, tp = confusion_matrix(y_test.view(-1), predicted.view(-1)).ravel()
print(tn, fp, fn, tp)
f_1 = (2 * tp) / (2 * tp + fp + fn)
print("F1 score:", f_1)
end_time = time.time()
end_time = end_time - start_time
times = str(timedelta(seconds=end_time)).split(':')
print(f'\nDuration: {times[0]} hours, {times[1]} minutes, {times[2]} seconds')
```